Отчёт по лабороторной работе №7

Дискретно-событийное моделирование

Селиванов Вячеслав Алексеевич

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

5 Выводы **44**

Список литературы **45**

Список иллюстраций

4.1 Инициализация проекта 8
4.2 Добавление пакетов 8
4.3 Модель M|M|c 9
4.4 Добавление графиков 9
4.5 Графики 9
4.6 Реализация модели Росса 10
4.7 Модель Росса с несколькими ремонтниками 10
4.8 Результат изменения количества машин 10
4.9 Мониторинг загрузки ремонтника 10
4.10 Построение графиков 11
4.11 Графики 11
4.12 Производные форматы 12
4.13 ПРоверка файлов ipynb 12

Список таблиц

1 Цель работы

Рассмотреть Дискретно-событийное моделирование на примере двух моделей.

2 Задание

Создать файлы с реализациями моделей, добавить графики и анализ влияния некоторых параметров.

3 Теоретическое введение

Модель M/M/c (по классификации Кендалла) — это система массового обслуживания со следующими свойствами: — M (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ. — M — время обслуживания каждой заявки распределено экспоненциально с параметром μ. — c — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

Модель Росса представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом. — В системе находятся ⊠ идентичных машин, которые постоянно работают и могут выходить из строя. — ⊠ машин находятся в резерве и готовы немедленно заменить любую отказавшую. — Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину. — Когда работающая машина ломается, происходит следующее: — Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся. — Сломанная машина отправляется в ремонт. — Если резерва нет, система падает (crash). Моделирование заканчивается. — После ремонта машина пополняет пул резервных (становится исправной и ждёт). — Требуется оценить среднее время до падения системы ⊠[⊠] при заданных распределениях наработки до отказа и времени ремонта.

4 Выполнение лабораторной работы

Инициализируем проект (рис. 4.1).

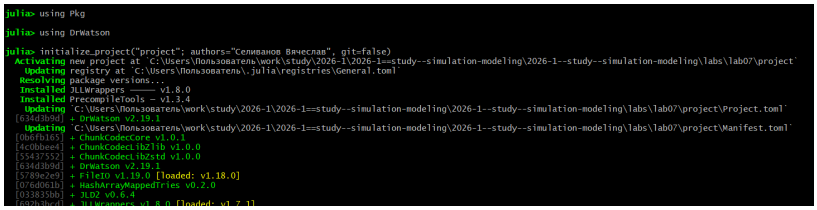
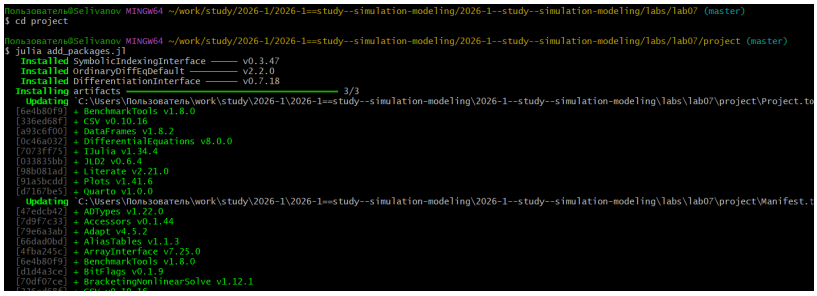
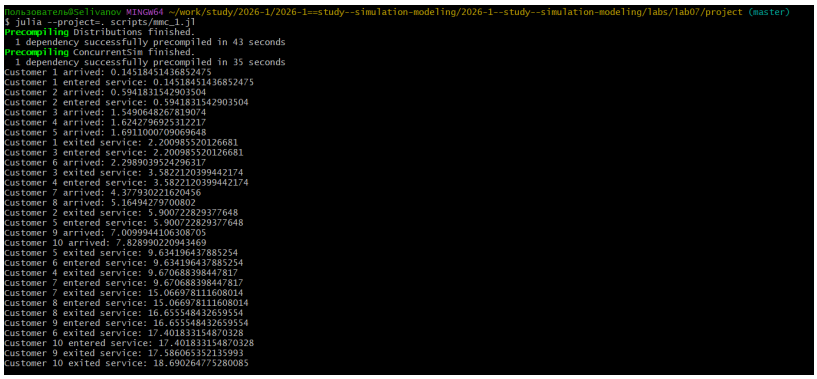


Рисунок 4.1: Инициализация проекта

Добавим в проект необходимые пакеты (рис. 4.2).

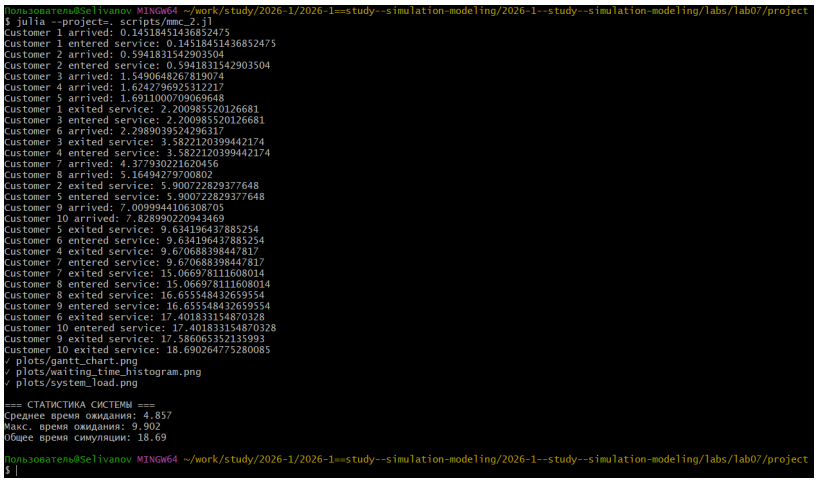

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07 (master)
$ cd project
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project (master)
$ julia add_packages.jl
Installed SymbolicIndexingInterface ─── v0.3.47
Installed OrdinaryDiffEqDefault ─────── v2.2.0
Installed DifferentiationInterface ─── v0.7.18
Installing artifacts ━━━



```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project (master)
$ julia --project=. scripts/mmc_1.jl
Precompiling Distributions finished.
  1 dependency successfully precompiled in 43 seconds
Precompiling ConcurrentSim finished.
  1 dependency successfully precompiled in 35 seconds
Customer 1 arrived: 0.1451845143685 2475
Customer 1 entered service: 0.1451845143685 2475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
```

Рисунок 4.3: Модель M|M|c

Добавим графики (рис. 4.4).



```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project
$ julia --project=. scripts/mmc_2.jl
Customer 1 arrived: 0.1451845143685 2475
Customer 1 entered service: 0.1451845143685 2475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
✓ plots/gantt_chart.png
✓ plots/waiting_time_histogram.png
✓ plots/system_load.png

=== СТАТИСТИКА СИСТЕМЫ ===
Среднее время ожидания: 4.857
Макс. время ожидания: 9.902
Общее время симуляции: 18.69

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project
$
```

Рисунок 4.4: Добавление графиков

Проанализируем графики, полученные на предыдущем шаге (рис. 4.5).

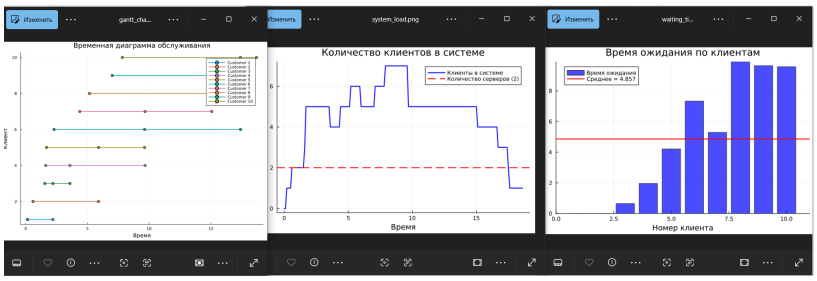


Рисунок 4.5: Графики

Реализуем модель Росса с заданными параметрами (рис. 4.6).

```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--s
$ julia --project=. scripts/ross_1.jl
At time 12715.718224958666: No more spares!
At time 37335.53567595007: No more spares!
At time 30844.62667837361: No more spares!
At time 1601.2524911974856: No more spares!
At time 824.1048708405848: No more spares!
Average crash time: 16664.247588264083

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--s
$
```

Рисунок 4.6: Реализация модели Росса

Добавим в модель Росса несколько ремонтников (рис. 4.7).

```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study
$ julia --project=. scripts/ross_2.jl
At time 87225.31575575111: No more spares!
At time 10542.406480466438: No more spares!
At time 112906.01822517307: No more spares!
At time 91683.47245483144: No more spares!
At time 78278.1405502697: No more spares!
Количество ремонтников: 5
Среднее время до краха: 76127.07069329834

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study
$
```

Рисунок 4.7: Модель Росса с несколькими ремонтниками

Изменим число машин и резервных машин (рис. 4.8).

```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07
$ julia --project=. scripts/ross_3.jl
At time 2203.233762967406: No more spares!
At time 45477.71831347564: No more spares!
At time 5758.162520917328: No more spares!
At time 10220.004165646245: No more spares!
At time 999.3783330647803: No more spares!
Average crash time: 12931.699419214281

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07
$
```

Рисунок 4.8: Результат изменения количества машин

Проведем мониторинг загрузки ремонтника, средней длины очереди на ремонт. (рис. 4.9).

```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project (master)
$ julia --project=. scripts/ross_4.jl
Запуск 5 симуляций с 1 ремонтником...

============================================================

--- Симуляция 1 ---
```

Рисунок 4.9: Мониторинг загрузки ремонтника

Построим графики изменения числа исправных машин во времени. (рис. 4.10).

```
Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/p
$ julia --project=. scripts/ross_5.jl
Запуск 5 симуляций...

--- Симуляция 1 ---
At time 12715.718224958666: No more spares!
--- Симуляция 2 ---
At time 37335.53567595007: No more spares!
--- Симуляция 3 ---
At time 30844.62667837361: No more spares!
--- Симуляция 4 ---
At time 1601.2524911974856: No more spares!
--- Симуляция 5 ---
At time 824.1048708405848: No more spares!

Average crash time: 16664.247588264083

=== ПОСТРОЕНИЕ ГРАФИКОВ ===
✓ plots/working_machines_time.png
✓ plots/queue_length_time.png
✓ plots/combined_state.png

=== СТАТИСТИКА ПОСЛЕДНЕЙ СИМУЛЯЦИИ ===
Время остановки: 824.105
Средняя длина очереди: 0.018
Макс. длина очереди: 2.0
Минимум исправных машин: 9.0
Среднее исправных машин: 12.86

Все графики сохранены в папку 'plots'

Пользователь@Selivanov MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/p
$
```

Рисунок 4.10: Построение графиков

Проанализируем графики. (рис. 4.11).

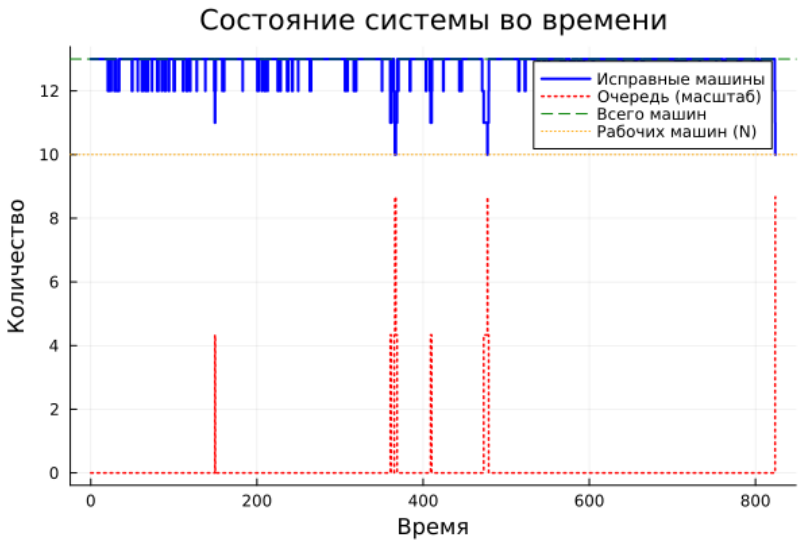


Рисунок 4.11: Графики

Сгенерируем необходимые производные форматы. (рис. 4.12).

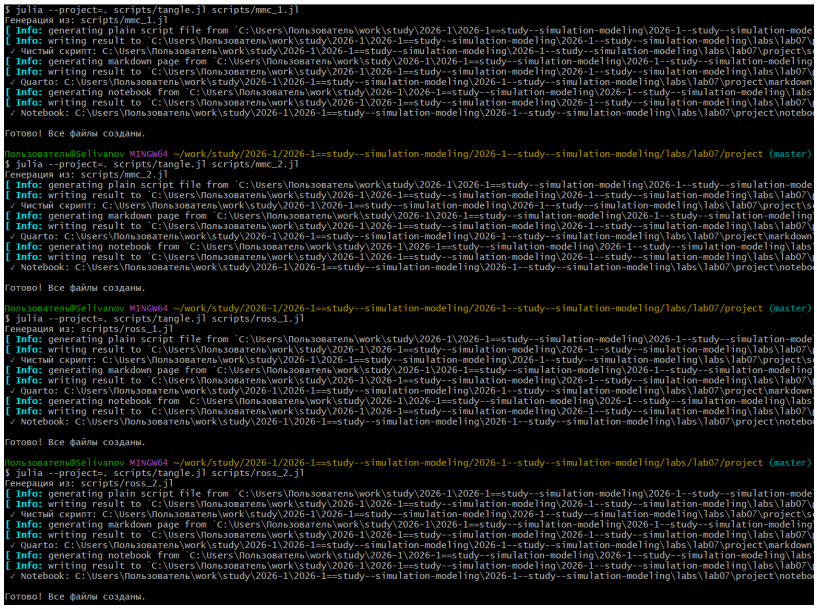


Рисунок 4.12: Производные форматы

Проверим работоспособность файлов .ipynb. (рис. 4.13).

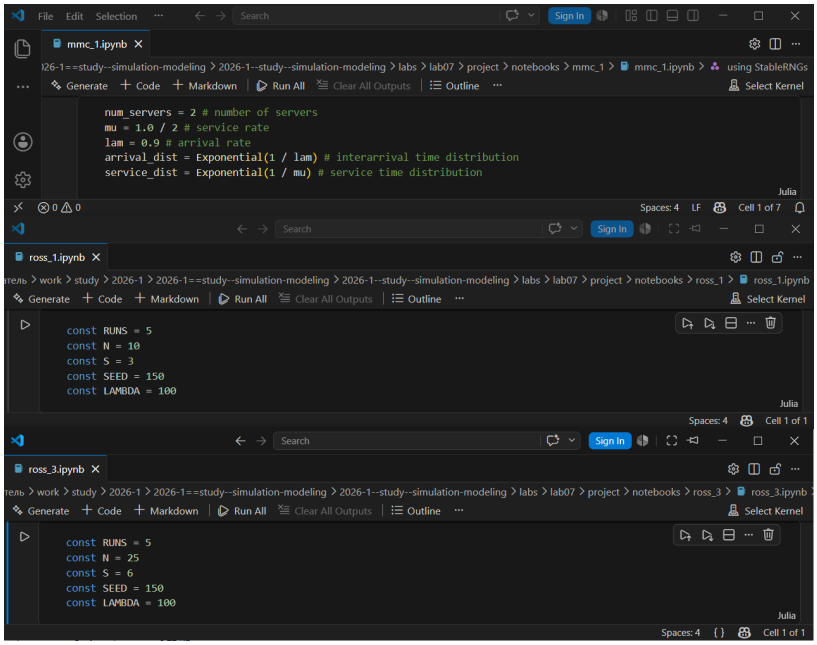


Рисунок 4.13: ПРоверка файлов ipynb

```
using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions

#set simulation parameters
rng = StableRNG(123)
num_customers = 10 # total number of customers generated
```

set queue parameters

```
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distributio
service_dist = Exponential(1 / mu) # service time distribution
```

define customer behavior

```
@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)
    @yield timeout(env, t_a) # customer arrives
    println("Customer $id arrived: ", now(env))
    @yield request(server) # customer starts service
    println("Customer $id entered service: ", now(env))
```

```
    @yield timeout(env, rand(rng, d_s)) # server is busy
    @yield unlock(server) # customer exits service
    println("Customer $id exited service: ", now(env))
end
```

setup and run simulation

```
function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dis
    end
    run(sim) # run simulation
end
setup_and_run()
```

```
using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions
using Plots
using Statistics

#set simulation parameters
rng = StableRNG(123)
num_customers = 10 # total number of customers generated
```

set queue parameters

```
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distributio
service_dist = Exponential(1 / mu) # service time distribution
```

Массивы для сбора данных

```
arrival_times = Vector{Float64}(undef, num_customers)
service_start_times = Vector{Float64}(undef, num_customers)
service_end_times = Vector{Float64}(undef, num_customers)
```

define customer behavior

```
@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)
    @yield timeout(env, t_a) # customer arrives
    arrival_times[id] = now(env)
    println("Customer $id arrived: ", now(env))
    @yield request(server) # customer starts service
    service_start_times[id] = now(env)
    println("Customer $id entered service: ", now(env))
    service_time = rand(rng, d_s)
```

```
    @yield timeout(env, service_time) # server is busy
    service_end_times[id] = now(env)
    println("Customer $id exited service: ", now(env))
    @yield unlock(server) # customer exits service
end
```

setup and run simulation

```
function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dis
    end
    run(sim) # run simulation
```

Создание графиков

```
    waiting_times = service_start_times .- arrival_times
```

1. Диаграмма Ганта

```
    p1 = plot(title="□□□□□□□□□ □□□□□□□□□ □□□□□□□□□□□□",
              xlabel="□□□□□", ylabel="□□□□□□", legend=:topright, size=
    for i in 1:num_customers
        plot!([arrival_times[i], service_start_times[i], service_en
              [i, i, i], marker=:circle, linewidth=2, label="Custom
    end
```

```
savefig(p1, "plots/gantt_chart.png")
println("□ plots/gantt_chart.png")
```

2. Гистограмма времени ожидания

```
p2 = bar(1:length(waiting_times), waiting_times,
         title="□□□□□ □□□□□□□□ □□ □□□□□□□□",
         xlabel="□□□□□ □□□□□□□", ylabel="□□□□□ □□□□□□□□",
         label="□□□□□ □□□□□□□□", color=:blue, alpha=0.7)
hline!([mean(waiting_times)], label="□□□□□□□ = $(round(mean(waiti
       color=:red, linewidth=2)
savefig(p2, "plots/waiting_time_histogram.png")
println("□ plots/waiting_time_histogram.png")
```

3. Динамика количества клиентов

```
time_points = 0:0.1:maximum(service_end_times)
n_clients = zeros(length(time_points))

for j in 1:length(time_points)
    t = time_points[j]
    count = 0
    for i in 1:num_customers
        if arrival_times[i] <= t && service_end_times[i] > t
            count += 1
        end
    end
    n_clients[j] = count
end
```

```
    p3 = plot(time_points, n_clients,
              title="□□□□□□□□□□ □□□□□□□□ □ □□□□□□□",
              xlabel="□□□□□", ylabel="□□□□□□□□□□ □□□□□□□□",
              label="□□□□□□□ □ □□□□□□□", linewidth=2, color=:blue)
    hline!([num_servers], linestyle=:dash, linewidth=2,
           label="□□□□□□□□□□ □□□□□□□□ ($num_servers)", color=:red)
    savefig(p3, "plots/system_load.png")
    println("□ plots/system_load.png")
```

Краткая статистика

```
    println("\n=== □□□□□□□□□□ □□□□□□□ ===")
    println("□□□□□□□ □□□□□ □□□□□□□□□: $(round(mean(waiting_times), digits
    println("□□□□. □□□□□ □□□□□□□□□: $(round(maximum(waiting_times), digi
    println("□□□□□ □□□□□ □□□□□□□□□□: $(round(maximum(service_end_times),
end

setup_and_run()
```

```
using ResumableFunctions
using ConcurrentSim

using Distributions
using Random
using StableRNGs

const RUNS = 5
const N = 10
const S = 3
```

```julia
const SEED = 150
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(rng, F))
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
        @yield request(repair_facility)
        @yield timeout(env, rand(rng, G))
```

```
        @yield unlock(repair_facility)
        @yield put!(spares, active_process(env))
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
```

```
    stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)
```

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using Statistics

const RUNS = 5
const N = 10   # □□□□□□□□□□ □□□□□
const S = 3    # □□□□□□□□□□ □□□□□□□□ □□□□□□
const SEED = 150
const LAMBDA = 100   # □□□□□□□□□□□□□ □□□□□□□
const MU = 1         # □□□□□□□□□□□□□ □□□□□□□
const NUM_REPAIR = 5   # □□□□□□□□□□ □□□□□□□□□□□

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)
```

```julia
@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(rng, F))  # □□□□□ □□ □□□□□□
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
        @yield request(repair_facility)  # □□□□□□ □□□□□□□□□□
        @yield timeout(env, rand(rng, G))  # □□□□□ □□□□□□□
        @yield unlock(repair_facility)     # □□□□□□□□□□□□ □□□□□□□□□□
        @yield put!(spares, active_process(env))  # □□□□□□□ □ □□□□□
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
```

```
    spares::Store{Process},
)
```

Запускаем N машин

```
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
```

Добавляем S запасных частей

```
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim, NUM_REPAIR)  # NUM_REPAIR ⌧⌧⌧⌧⌧⌧
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    return stop_time
end

results = Float64[]
```

```
for i = 1:RUNS
    push!(results, sim_repair())
end

println("□□□□□□□□□□ □□□□□□□□□□□: $NUM_REPAIR")
println("□□□□□□□ □□□□□ □□ □□□□□: ", sum(results)/RUNS)
```

```
using ResumableFunctions
using ConcurrentSim

using Distributions
using Random
using StableRNGs

const RUNS = 5
const N = 25
const S = 6
const SEED = 150
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
```

```
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(rng, F))
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
        @yield request(repair_facility)
        @yield timeout(env, rand(rng, G))
        @yield unlock(repair_facility)
        @yield put!(spares, active_process(env))
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    for i = 1:N
```

```
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)
```

```
using ResumableFunctions
using ConcurrentSim
```

```julia
using Distributions
using Random
using StableRNGs
using Statistics

const RUNS = 5
const N = 10
const S = 3
const SEED = 150
const LAMBDA = 100
const MU = 1
const NUM_REPAIR = 1  # [illegible] [illegible]

const rng = StableRNG(42)
const F = Exponential(LAMBDA)
const G = Exponential(MU)
```

Глобальные переменные для мониторинга

```julia
queue_lengths = Float64[]
queue_times = Float64[]
repair_busy_time = 0.0  # [illegible] [illegible] [illegible]
total_repair_time = 0.0  # [illegible] [illegible] [illegible]
last_state_change = 0.0  # [illegible] [illegible] [illegible] [illegible]

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
```

```
)
    global repair_busy_time, last_state_change

    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(rng, F))
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
```

Запрос ремонтника и запись длины очереди

```
        current_queue = length(repair_facility.queue)
        push!(queue_times, now(env))
        push!(queue_lengths, current_queue)

        @yield request(repair_facility)
```

Ремонт

```
        repair_time = rand(rng, G)
        @yield timeout(env, repair_time)
```

```
        @yield unlock(repair_facility)
        @yield put!(spares, active_process(env))
    end
end

@resumable function monitor_system(
    env::Environment,
    repair_facility::Resource
)
    while true
        @yield timeout(env, 0.1)  # □□□□□□□□□ □□□□□□ 0.1 □□□□□□□
        current_queue = length(repair_facility.queue)
        push!(queue_times, now(env))
        push!(queue_lengths, current_queue)
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
```

Запускаем мониторинг

```
    @process monitor_system(env, repair_facility)

    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
```

```
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    global queue_lengths, queue_times, repair_busy_time, last_state
```

Сбрасываем данные

```
    queue_lengths = Float64[]
    queue_times = Float64[]
    repair_busy_time = 0.0
    last_state_change = 0.0

    sim = Simulation()
    repair_facility = Resource(sim, NUM_REPAIR)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
```

Расчет метрик

```
    avg_queue_length = length(queue_lengths) > 0 ? mean(queue_lengt
    max_queue_length = length(queue_lengths) > 0 ? maximum(queue_le
```

Расчет загрузки ремонтника через статистику ресурса ConcurrentSim не дает прямой доступ к времени занятости, оцениваем косвенно

```
    utilization = (1 - (length(repair_facility.queue) / (NUM_REPAIR
```

Альтернативный расчет загрузки через количество обращений

```
    total_services = length(repair_facility.server)  # [redacted] [redacted]

    println("\n=== [redacted] [redacted] ===")
    println("[redacted]: $(round(stop_time, digits=3))")
    println("[redacted]: $msg")
    println("\n=== [redacted] [redacted] ===")
    println("[redacted] [redacted]: $(round(avg_queue_length, digits=3))
    println("[redacted] [redacted]: $max_queue_length")
    println("[redacted]: $(length(queue_lengths))")
    println("\n=== [redacted] [redacted] ===")
    println("[redacted]: $NUM_REPAIR")
    println("[redacted]: $(round(utilization, digits=2))%")

    return stop_time, avg_queue_length, max_queue_length, utilizati
end
```

Запуск нескольких симуляций

```
results_time = Float64[]
results_queue = Float64[]
results_maxqueue = Float64[]
results_util = Float64[]
```

```
println("□□□□□□ $(RUNS) □□□□□□□□□ □ 1 □□□□□□□□□□□...\n")
println("="^60)

for i = 1:RUNS
    println("\n--- □□□□□□□□□ $i ---")
    stop_time, avg_q, max_q, util = sim_repair()
    push!(results_time, stop_time)
    push!(results_queue, avg_q)
    push!(results_maxqueue, max_q)
    push!(results_util, util)
end
```

Итоговая статистика

```
println("\n" * "="^60)
println("□□□□□□□□ □□□□□□□□□□ □□ $(RUNS) □□□□□□□□ (1 □□□□□□□□□)")
println("="^60)

println("\n=== □□□□□ □□ □□□□□ ===")
println("□□□□□□□: $(round(mean(results_time), digits=3))")
println("□□. □□□□□□□□□□: $(round(std(results_time), digits=3))")
println("□□□□□□□: $(round(minimum(results_time), digits=3))")
println("□□□□□□□□: $(round(maximum(results_time), digits=3))")

println("\n=== □□□□□□□ □□□□□ □□□□□□□ ===")
println("□□□□□□□: $(round(mean(results_queue), digits=3))")
println("□□. □□□□□□□□□□: $(round(std(results_queue), digits=3))")
println("□□□□□□□: $(round(minimum(results_queue), digits=3))")
println("□□□□□□□□: $(round(maximum(results_queue), digits=3))")
```

```
println("\n=== □□□□□□□□□□□□ □□□□□ □□□□□□□ ===")
println("□□□□□□□: $(round(mean(results_maxqueue), digits=3))")
println("□□. □□□□□□□□□□: $(round(std(results_maxqueue), digits=3))")

println("\n=== □□□□□□□□ □□□□□□□□□□ ===")
println("□□□□□□□: $(round(mean(results_util), digits=2))%")
println("□□. □□□□□□□□□□: $(round(std(results_util), digits=2))%")
println("□□□□□□□: $(round(minimum(results_util), digits=2))%")
println("□□□□□□□□: $(round(maximum(results_util), digits=2))%")

println("\n" * "="^60)
println("□□□□□□□□□:")
println("□□□ 1 □□□□□□□□□□ □□□□□□□ □□□□□ □□□□□ = $(round(mean(results_time)
println("□□□□□□□ □□□□□□□ = $(round(mean(results_queue), digits=3))")
println("□□□□□□□□ = $(round(mean(results_util), digits=2))%")
```

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using Plots
using Statistics

const RUNS = 5
const N = 10
const S = 3
const SEED = 150
```

```julia
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)
```

Глобальные переменные для сбора данных

```julia
all_working_machines = Vector{Vector{Float64}}()
all_working_times = Vector{Vector{Float64}}()
all_queue_lengths = Vector{Vector{Float64}}()
all_queue_times = Vector{Vector{Float64}}()
```

Счетчик машин в очереди

```julia
queue_counter = 0

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)
    global queue_counter
```

```
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
```

Машина работает

```
        @yield timeout(env, rand(rng, F))
```

Поломка - уменьшаем исправные машины

```
        push!(working_times, now(env))
        push!(working_machines, working_machines[end] - 1)

        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
```

Запись очереди перед запросом ремонтника

```
        push!(queue_times, now(env))
        push!(queue_lengths, queue_counter)

        queue_counter += 1  # [illegible] [illegible] [illegible]
        @yield request(repair_facility)
        queue_counter -= 1  # [illegible] [illegible] [illegible]
```

```
        @yield timeout(env, rand(rng, G))
        @yield unlock(repair_facility)
```

Ремонт завершен - увеличиваем исправные машины

```
        push!(working_times, now(env))
        push!(working_machines, working_machines[end] + 1)

        @yield put!(spares, active_process(env))
    end
end

@resumable function monitor_system(
    env::Environment,
    repair_facility::Resource,
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)
    global queue_counter

    while true
        @yield timeout(env, 0.1)
```

Запись текущей длины очереди

```
        push!(queue_times, now(env))
        push!(queue_lengths, queue_counter)
```

Запись исправных машин

```
        push!(working_times, now(env))
        push!(working_machines, working_machines[end])
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)
```

Запускаем мониторинг

```
    @process monitor_system(env, repair_facility, working_machines,
                            queue_lengths, queue_times, total_machin

    for i = 1:N
        proc = @process machine(env, repair_facility, spares, worki
                                working_times, queue_lengths, queue_
        @yield interrupt(proc)
```

```
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares, worki
                                working_times, queue_lengths, queue_
        @yield put!(spares, proc)
    end
end

function sim_repair()
    global queue_counter
    queue_counter = 0

    total_machines = N + S
    working_machines = [Float64(total_machines)]
    working_times = [0.0]
    queue_lengths = Float64[]
    queue_times = Float64[]

    sim = Simulation()
    repair_facility = Resource(sim)  # 1 ⌧⌧⌧⌧⌧⌧⌧⌧
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares, working_machine
                       queue_lengths, queue_times, total_machines)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
```

Сохраняем данные для графиков

```
    push!(all_working_machines, copy(working_machines))
    push!(all_working_times, copy(working_times))
    push!(all_queue_lengths, copy(queue_lengths))
    push!(all_queue_times, copy(queue_times))

    return stop_time, working_machines, working_times, queue_length
end
```

Запуск симуляций и сбор данных

```
results = Float64[]
all_stop_times = Float64[]

println("□□□□□□ $(RUNS) □□□□□□□□□...\n")

for i = 1:RUNS
    println("--- □□□□□□□□□ $i ---")
    stop_time, _, _, _, _ = sim_repair()
    push!(results, stop_time)
    push!(all_stop_times, stop_time)
end

println("\nAverage crash time: ", sum(results)/RUNS)
```

Построение графиков по последней симуляции

```
println("\n=== □□□□□□□□□□ □□□□□□□□ ===")
```

Создаем папку для графиков

```
if !isdir("plots")
    mkdir("plots")
end
```

Берем данные последней симуляции

```
working_machines = all_working_machines[end]
working_times = all_working_times[end]
queue_lengths = all_queue_lengths[end]
queue_times = all_queue_times[end]
total_machines = N + S
```

График 1: Количество исправных машин во времени

```
p1 = plot(title="□□□□□□□□□□ □□□□□□□□□□ □□□□□ □□ □□□□□□□",
          xlabel="□□□□□", ylabel="□□□□□□□□□□ □□□□□□□□□□ □□□□□",
          legend=:topright, linewidth=2)
```

Подготовка данных для ступенчатого графика

```
plot_times = []
plot_values = []
for i in 1:length(working_times)-1
    push!(plot_times, working_times[i])
    push!(plot_values, working_machines[i])
    push!(plot_times, working_times[i+1])
    push!(plot_values, working_machines[i])
end

plot!(plot_times, plot_values, label="□□□□□□□□□ □□□□□□", color=:blue, s
```

```
hline!([total_machines], linestyle=:dash, label="□□□□□ □□□□□ ($total_m
hline!([N], linestyle=:dot, label="□□□□□□□ □□□□□ (N=$N)", color=:orang
savefig(p1, "plots/working_machines_time.png")
println("□ plots/working_machines_time.png")
```

График 2: Длина очереди на ремонт

```
p2 = plot(title="□□□□□ □□□□□□□ □□ □□□□□□",
          xlabel="□□□□□", ylabel="□□□□□ □□□□□□□",
          legend=:topright, linewidth=2)
```

Подготовка данных для ступенчатого графика очереди

```
if length(queue_times) > 0
    queue_times_plot = []
    queue_vals_plot = []
    for i in 1:length(queue_times)-1
        push!(queue_times_plot, queue_times[i])
        push!(queue_vals_plot, queue_lengths[i])
        push!(queue_times_plot, queue_times[i+1])
        push!(queue_vals_plot, queue_lengths[i])
    end

    plot!(queue_times_plot, queue_vals_plot, label="□□□□□ □□□□□□□", co
    if length(queue_lengths) > 0
        hline!([mean(queue_lengths)], linestyle=:dash,
               label="□□□□□□□ = $(round(mean(queue_lengths), digits=2
    end
end
```

```
savefig(p2, "plots/queue_length_time.png")
println("□ plots/queue_length_time.png")
```

График 3: Совмещенный график

```
p3 = plot(title="□□□□□□□□□ □□□□□□□ □□ □□□□□□□",
          xlabel="□□□□□", ylabel="□□□□□□□□□□",
          legend=:topright)
```

Исправные машины

```
plot!(plot_times, plot_values, label="□□□□□□□□□ □□□□□□", color=:blue, s
```

Очередь (масштабируем для наглядности)

```
if length(queue_lengths) > 0 && maximum(queue_lengths) > 0
    scale_factor = total_machines / (maximum(queue_lengths) + 1)
    queue_scaled = queue_vals_plot .* scale_factor
    plot!(queue_times_plot, queue_scaled, label="□□□□□□□ (□□□□□□□)", co
end

hline!([total_machines], linestyle=:dash, label="□□□□□ □□□□□", color=
hline!([N], linestyle=:dot, label="□□□□□□□ □□□□□ (N)", color=:orange)
savefig(p3, "plots/combined_state.png")
println("□ plots/combined_state.png")
```

Дополнительная статистика

```
println("\n=== □□□□□□□□□□ □□□□□□□□□ □□□□□□□□□ ===")
println("□□□□□ □□□□□□□□□: $(round(all_stop_times[end], digits=3))")
println("□□□□□□□ □□□□□ □□□□□□□: $(round(mean(queue_lengths), digits=3))"
```

```
println("□□□□. □□□□□ □□□□□□□: $(round(maximum(queue_lengths), digits=0)
println("□□□□□□□ □□□□□□□□□ □□□□□: $(round(minimum(working_machines), dig
println("□□□□□□□ □□□□□□□□□ □□□□□: $(round(mean(working_machines), digits

println("\n□□□ □□□□□□□ □□□□□□□□□ □ □□□□□ 'plots'")
```

5 Выводы

В ходе данной лабораторной работы я познакомился и поработал с Дискретно-событийным моделированием на примере двух моделей M|M|c и Росса.

Список литературы